OCOM5300M Artificial Intelligence Project Performance

In [ ]:
```python
# Custom Classes
from captions import Captions
from evaluation import BLEU_Evaluation
from sentence_transformers import SentenceTransformer
from rouge_score import rouge_scorer

# Deep learning Functionality
import torch  # PyTorch library

# Data Handling And Analysis
import pandas as pd  # data manipulation and analysis using DataFrame structures
import numpy as np  # numerical operations and array handling using arrays

# Natural Language Processing And Evaluation
import nltk
from nltk.translate.bleu_score import corpus_bleu, SmoothingFunction  # BLEU score evaluation
from nltk.translate.meteor_score import meteor_score
import re

# Visualization
import matplotlib.pyplot as plt  # plotting and visualizing results
import seaborn as sns  # visualisation library for statistical graphics

# Utilities
import os  # file and directory operations
import random # random number generation
```

In [ ]:
```python
# suppress Intel specific Math Kernel Library (MKL) warnings
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'
```

In [ ]:
```python
#detect device (CPU/GPU)
device = torch.device('cpu')
```

In [ ]:
```python
# display the complete content of DataFrame columns without truncation
pd.set_option('display.max_colwidth', None);
```

In [ ]:
```
# seed for reproducibility
seed = 42
```

In [ ]:
```
# Python random seed
random.seed(seed)
```

In [ ]:
```
# NumPy random seed
np.random.seed(seed)
```

In [ ]:
```
# PyTorch random seed
torch.manual_seed(seed)
```

Ground Truths

In [ ]:
```
# test captions filename
test_captions_filename = "ROCOv2/test_captions.csv"
```

In [ ]:
```
# instantiate test captions object
test_captions = Captions()
```

In [ ]:
```
# load test captions from file
test_caption_data = test_captions.load_captions(test_captions_filename)
```

In [ ]:
```
# clean captions
test_captions.clean_captions();
```

In [ ]:
```
# confirm number of captions
len(test_caption_data)
```

In [ ]:
```
test_caption_data.head()
```

Load Generated Captions

In [ ]:
```
# select model architecture
architecture = "Recurrent"  # Recurrent, Transformer or VLM
```

In [ ]:
```python
# load previously generated captions from CSV
if architecture == "Recurrent":
    generated_captions = pd.read_csv("LSTM_generated_captions.csv")

elif architecture == "Transformer":
    generated_captions = pd.read_csv("Transformer_generated_captions.csv")

elif architecture == "VLM":
    generated_captions = pd.read_csv("VLM_generated_captions.csv")
```

In [ ]:
```python
# display subset of loaded captions
generated_captions.head()
```

Diagnostics

In [ ]:
```python
# sample, i.e., for final run all images in the test set
sample_image_ids = (test_captions.captions["ID"].drop_duplicates().tolist())
```

In [ ]:
```python
# filter captions
sample = test_captions.captions.loc[test_captions.captions["ID"].isin(sample_image_ids), ["ID", "cleaned_caption"]]
```

In [ ]:
```python
def caption_statistics(df, column_name):

    """
    Calculate caption length statistics

    Args:
       df (pandas.DataFrame): dataframe containing caption data
       column_name (str): name of the column in the dataframe that contains captions

    Returns:
       tuple: A tuple containing two elements:
           - statistics (pandas.DataFrame): dataframe with statistical measures
             (min, median, mean, max, standard deviation) realting to caption lengths
           - caption_lengths (pandas.Series): series containing the word count for each caption
    """

    try:
```

```
        # summary statistics
        statistics = pd.DataFrame(columns=['min', 'median', 'mean', 'max', 'std'])

        # return the number of words in the caption and statistical analysis
        caption_lengths = df[column_name].apply(lambda text: len(str(text).split()))

        # add caption lengths statistics, i.e., minimum, median, mean, maximum and standard deviation
        statistics.loc[len(statistics)] = [round(caption_lengths.min(), 2),
                                           round(caption_lengths.median(), 2),
                                           round(caption_lengths.mean(), 2),
                                           round(caption_lengths.max(), 2),
                                           round(caption_lengths.std(), 2)]

        # return caption length statistics and individual caption lengths
        return statistics, caption_lengths

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [ ]:

```
def format_statistics_table(df_generated_statistics, df_ground_truth_statistics):

    """
    Create HTML table to display and compare caption statistics from different caption generation methods,
    i.e., generated captions and ground truth.

    Args:
        df_generated_statistics (pandas.DataFrame): dataFrame containing statistical measures
                                                    for generated captions.
        df_ground_truth_statistics (pandas.DataFrame): dataFrame containing statistical measures
                                                       for ground truth captions.

    Returns:
        str: HTML-formatted string representing a table with caption statistics.
             The table includes rows for generated captions and ground truth,
             with columns for statistical measures (Min, Median, Mean, Max, Std).
    """

    try:

        # table header
        html_table = """
```

```
        <table border="1" class="dataframe">
          <thead>
            <tr style="text-align: center;">
              <th>Caption Type</th>
              <th>Min</th>
              <th>Median</th>
              <th>Mean</th>
              <th>Max</th>
              <th>Std</th>
            </tr>
          </thead>
          <tbody>
        """

        # format and add generated captions row
        min_val = f"{df_generated_statistics['min'].iloc[0]:.2f}" if 'min' in df_generated_statistics else ""
        median_val = f"{df_generated_statistics['median'].iloc[0]:.2f}" if 'median' in df_generated_statistics else
""
        mean_val = f"{df_generated_statistics['mean'].iloc[0]:.2f}" if 'mean' in df_generated_statistics else ""
        max_val = f"{df_generated_statistics['max'].iloc[0]:.2f}" if 'max' in df_generated_statistics else ""
        std_val = f"{df_generated_statistics['std'].iloc[0]:.2f}" if 'std' in df_generated_statistics else ""

        html_table += f"""
        <tr>
          <td>Generated Captions</td>
          <td>{min_val}</td>
          <td>{median_val}</td>
          <td>{mean_val}</td>
          <td>{max_val}</td>
          <td>{std_val}</td>
        </tr>
        """

        # format and add ground truth row

        min_val = f"{df_ground_truth_statistics['min'].iloc[0]:.2f}" if 'min' in df_ground_truth_statistics else ""
        median_val = f"{df_ground_truth_statistics['median'].iloc[0]:.2f}" if 'median' in
df_ground_truth_statistics else ""
        mean_val = f"{df_ground_truth_statistics['mean'].iloc[0]:.2f}" if 'mean' in df_ground_truth_statistics else
""
        max_val = f"{df_ground_truth_statistics['max'].iloc[0]:.2f}" if 'max' in df_ground_truth_statistics else ""
        std_val = f"{df_ground_truth_statistics['std'].iloc[0]:.2f}" if 'std' in df_ground_truth_statistics else ""
```

```
        html_table += f"""
        <tr>
          <td>Ground Truth</td>
          <td>{min_val}</td>
          <td>{median_val}</td>
          <td>{mean_val}</td>
          <td>{max_val}</td>
          <td>{std_val}</td>
        </tr>
        """

        # Close the table
        html_table += """
          </tbody>
        </table>
        """

        return html_table

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [ ]:

```
def density_plots(generated_lengths, groundtruth_lengths):

    """
    Creates and displays density plots comparing the distribution of caption lengths from different caption
generation methods,
    i.e., generated captions and ground truth.

    Args:
        generated_lengths (pandas.Series or array-like): caption lengths from generated captions
        groundtruth_lengths (pandas.Series or array-like): caption lengths from ground truth data

    """

    try:

        plt.figure(figsize=(12, 6))
        sns.set_style("whitegrid")
```

```
        # plot KDE for each distribution
        sns.kdeplot(generated_lengths, label='Generated Captions', fill=True, alpha=0.3)
        sns.kdeplot(groundtruth_lengths, label='Ground Truth', fill=True, alpha=0.3)

        # add vertical lines for averages (means)
        plt.axvline(generated_lengths.mean(), color='orange', linestyle='--',
                    label=f'Generated Mean: {generated_lengths.mean():.2f}')
        plt.axvline(groundtruth_lengths.mean(), color='green', linestyle='--',
                    label=f'GT Mean: {groundtruth_lengths.mean():.2f}')

        # configure title, axis labels, legend and layout
        plt.title('Distribution of Caption Lengths')
        plt.xlabel('Caption Length (Words)')
        plt.ylabel('Density')
        plt.legend()
        plt.tight_layout()
        plt.show()

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [ ]:
```
# calculate generated caption statistics
generated_statistics, generated_lengths = caption_statistics(generated_captions, "generated_caption")
```

In [ ]:
```
# calculate ground truth statistics
groundtruth_statistics, groundtruth_lengths = caption_statistics(sample, "cleaned_caption")
```

In [ ]:
```
# create output
from IPython.display import HTML, display
html_output = format_statistics_table(generated_statistics, groundtruth_statistics)
```

In [ ]:
```
# display statistics
HTML(html_output)
```

In [ ]:
```
# create density plots for caption lenghts
density_plots(generated_lengths, groundtruth_lengths)
```

Caption Evaluation Using BLEU Score

In [ ]:
```
# extract captions
test_set_captions = test_captions.captions.loc[test_captions.captions["ID"].isin(sample_image_ids), ["ID",
"cleaned_caption"]]
```

In [ ]:
```
# instantiate evaluator
BLEU_evaluator = BLEU_Evaluation()
```

In [ ]:
```
BLEU_generated_captions = generated_captions
```

In [ ]:
```
# merge test set and generated captions
BLEU_reference_generated_captions = pd.merge(test_set_captions, BLEU_generated_captions, on='ID', how='inner')
```

In [ ]:
```
# calculate BLEU scores
BLEU_results = BLEU_evaluator.calculate_weighted_bleu_scores(BLEU_reference_generated_captions)
```

In [ ]:
```
# display BLEU scores related to sample set
BLEU_results[BLEU_results['ID'].isin(sample_image_ids)]
```

In [ ]:
```
# display BLEU score histogram
BLEU_evaluator.bleu_scores_histogram()
```

In [ ]:
```
def calculate_corpus_bleu(captions_dataframe):
    """
    Calculate and display corpus BLEU-1 to BLEU-4 scores using reference and generated captions, i.e. calculate
statistics across all test captions

    Args:
        captions_dataframe (pd.DataFrame): DataFrame containing image IDs, reference captions and generated
captions

    Returns:
        list: corpus BLEU-1 to BLEU-4 scores
    """

    try:

        # create empty lists to store reference and generated captions for corpus-level evaluation
        corpus_references = []
```

```
        corpus_hypotheses = []

        # group captions by image ID
        for image_id, group in captions_dataframe.groupby('ID'):

            # convert reference captions to lowercase word tokens
            references = [reference.lower().split() for reference in group['cleaned_caption']]

            # convert generated caption to lowercase word tokens
            hypothesis = group['generated_caption'].iloc[0].lower().split()

            # store reference and generated captions
            corpus_references.append(references)
            corpus_hypotheses.append(hypothesis)

        # calculate corpus BLEU-1 to BLEU-4 scores
        corpus_bleu_scores = corpus_bleu(corpus_references, corpus_hypotheses, weights=[(1, 0, 0, 0),
                                                                                        (0.5, 0.5, 0, 0),
                                                                                        (1/3, 1/3, 1/3, 0), #
prevent approximation
                                                                                        (0.25, 0.25, 0.25, 0.25)],

smoothing_function=SmoothingFunction().method1)


        # create BLEU score labels
        bleu_labels = ['BLEU-1', 'BLEU-2', 'BLEU-3', 'BLEU-4']

        # plot corpus BLEU scores
        plt.figure(figsize=(8, 5))
        plt.bar(bleu_labels, corpus_bleu_scores)
        plt.title('Corpus BLEU Scores')
        plt.xlabel('BLEU Metric')
        plt.ylabel('BLEU Score')
        plt.ylim(0, 1)
        plt.grid(axis='y')
        plt.tight_layout()
        plt.show()

        return corpus_bleu_scores
```

```
    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [ ]:
```
# calculate corpus BLEU-1 to BLEU-4 scores for the generated captions
corpus_bleu_scores = calculate_corpus_bleu(BLEU_reference_generated_captions)
```

Rouge Score

In [ ]:
```
# merge ground-truth and generated captions by image ID
ROUGE_reference_generated_captions = pd.merge(
    test_set_captions,
    generated_captions,
    on="ID",
    how="inner",
    validate="many_to_one"
)
```

In [ ]:
```
# initialise ROUGE-L evaluator
rouge_evaluator = rouge_scorer.RougeScorer(
    ["rougeL"],
    use_stemmer=False
)
```

In [ ]:
```
# calculate ROUGE-L for each reference/generated caption pair
rouge_scores = [
    rouge_evaluator.score(reference, generated)["rougeL"]
    for reference, generated in zip(
        ROUGE_reference_generated_captions["cleaned_caption"],
        ROUGE_reference_generated_captions["generated_caption"]
    )
]
```

In [ ]:
```
# store ROUGE-L precision, recall and F1
ROUGE_reference_generated_captions["ROUGE-L Precision"] = [
    score.precision for score in rouge_scores
]
```

In [ ]:
```
ROUGE_reference_generated_captions["ROUGE-L Recall"] = [
    score.recall for score in rouge_scores
```

```
]
```

In [ ]:
```
ROUGE_reference_generated_captions["ROUGE-L F1"] = [
    score.fmeasure for score in rouge_scores
]
```

In [ ]:
```
# select reference with highest ROUGE-L F1 per image
best_reference_indices = (
    ROUGE_reference_generated_captions
    .groupby("ID")["ROUGE-L F1"]
    .idxmax()
)
```

In [ ]:
```
# retain precision, recall and F1 from that same reference
ROUGE_results = (
    ROUGE_reference_generated_captions
    .loc[
        best_reference_indices,
        [
            "ID",
            "ROUGE-L Precision",
            "ROUGE-L Recall",
            "ROUGE-L F1"
        ]
    ]
    .reset_index(drop=True)
)
```

In [ ]:
```
# display ROUGE-L summary statistics
print(
    ROUGE_results[
        ["ROUGE-L Precision", "ROUGE-L Recall", "ROUGE-L F1"]
    ].describe()
)
```

In [ ]:
```
# display mean evaluation scores
print("\nMean ROUGE-L Scores")

for metric in [
    "ROUGE-L Precision",
```

```
    "ROUGE-L Recall",
    "ROUGE-L F1"
]:
    print(f"{metric}: {ROUGE_results[metric].mean():.4f}")
```

In [ ]:
```
# display ROUGE-L score distributions
for metric in [
    "ROUGE-L Precision",
    "ROUGE-L Recall",
    "ROUGE-L F1"
]:
    plt.figure(figsize=(10, 5))
    plt.hist(ROUGE_results[metric], bins=40)
    plt.title(f"{metric} Distribution")
    plt.xlabel("Score")
    plt.ylabel("Number of Images")
    plt.tight_layout()
    plt.show()
```

In [ ]:
```
# display ROUGE-L results for each image
display(ROUGE_results)
```

Meteor

In [ ]:
```
# merge ground-truth and generated captions by image ID
METEOR_reference_generated_captions = pd.merge(
    test_set_captions,
    generated_captions,
    on="ID",
    how="inner",
    validate="many_to_one"
)
```

In [ ]:
```
# tokenise captions consistently
def meteor_tokenize(caption):
    return re.findall(r"[a-z0-9]+", caption.lower())
```

In [ ]:
```
# calculate METEOR score for each image
meteor_results = []
```

In [ ]:
```python
for image_id, group in METEOR_reference_generated_captions.groupby("ID"):

    # retrieve generated caption
    generated = group["generated_caption"].iloc[0]

    # retrieve and tokenise all ground-truth captions
    references = [
        meteor_tokenize(ref)
        for ref in group["cleaned_caption"]
        if isinstance(ref, str) and ref.strip()
    ]

    # calculate METEOR using the best matching reference
    if references and isinstance(generated, str) and generated.strip():
        score = meteor_score(
            references,
            meteor_tokenize(generated)
        )

        meteor_results.append({
            "ID": image_id,
            "METEOR": score
        })
```

In [ ]:
```python
# store results in DataFrame
METEOR_results = pd.DataFrame(meteor_results)
```

In [ ]:
```python
# display METEOR summary statistics
print(METEOR_results["METEOR"].describe())
```

In [ ]:
```python
# display mean METEOR score
print(
    f"\nMean METEOR Score: "
    f"{METEOR_results['METEOR'].mean():.4f}"
)
```

In [ ]:
```python
# display METEOR score distribution
plt.figure(figsize=(10, 5))

plt.hist(
```

```
    METEOR_results["METEOR"],
    bins=40
)

plt.title("METEOR Score Distribution")
plt.xlabel("METEOR Score")
plt.ylabel("Number of Images")

plt.tight_layout()
plt.show()
```

In [ ]:
```
# display METEOR results for each image
display(METEOR_results)
```

Caption Evaluation Using Cosine Similarity

In [ ]:
```
# load pretrained sentence embedding model
sentence_model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2", device="cpu")
```

In [ ]:
```
# merge ground-truth and generated captions by image ID
CS_reference_generated_captions = pd.merge(
    test_set_captions,
    generated_captions,
    on="ID",
    how="inner",
    validate="many_to_one"
)
```

In [ ]:
```
# verify every generated image has a reference caption
missing_ids = set(generated_captions["ID"]) - set(
    CS_reference_generated_captions["ID"]
)
```

In [ ]:
```
# create sentence embeddings for ground-truth captions
reference_embeddings = sentence_model.encode(
    CS_reference_generated_captions["cleaned_caption"].tolist(),
    batch_size=64,
    normalize_embeddings=True,
    convert_to_numpy=True,
```

```
    show_progress_bar=True
)
```

In [ ]:
```
# create sentence embeddings for generated captions
generated_embeddings = sentence_model.encode(
    CS_reference_generated_captions["generated_caption"].tolist(),
    batch_size=64,
    normalize_embeddings=True,
    convert_to_numpy=True,
    show_progress_bar=True
)
```

In [ ]:
```
# calculate cosine similarity for corresponding captions
CS_reference_generated_captions["cosine_similarity"] = np.sum(
    reference_embeddings * generated_embeddings,
    axis=1
).clip(-1, 1)
```

In [ ]:
```
# retain best matching reference per image
CS_results = (
    CS_reference_generated_captions
    .groupby("ID", as_index=False)["cosine_similarity"]
    .max()
)
```

In [ ]:
```
# display summary statistics
print(CS_results["cosine_similarity"].describe())
```

In [ ]:
```
# display distribution
plt.figure(figsize=(10, 5))
plt.hist(
    CS_results["cosine_similarity"],
    bins=40
)
plt.title("Generated Caption Cosine Similarity")
plt.xlabel("Cosine Similarity")
plt.ylabel("Number of Images")
plt.tight_layout()
plt.show()
```

In [ ]:
```
# display individual image results
display(CS_results)
```

References

Hugging Face (2026) all-MiniLM-L6-v2: Pretrained Sentence Embedding Model. Available At https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Natural Language Toolkit (NLTK) (2026) NLTK Python Package – BLEU and METEOR Evaluation. Available At https://pypi.org/project/nltk/

Natural Language Toolkit (NLTK) (2026) METEOR Score: API Documentation. Available At https://www.nltk.org/api/nltk.translate.meteor_score.html

Natural Language Toolkit (NLTK) (2026) Installing NLTK Data – WordNet and Linguistic Resources. Available At https://www.nltk.org/data.html

OCOM5100M Programming For Data Science

OCOM5101M Data Science

OCOM5200M Machine Learning

OCOM5203M Deep Learning

OCOM5203M Deep Learning Module, Summative Assessment 2

OCOM5204M Data Mining And Analytics

Project Jupyter, Jupyter Notebook documentation. Available At: https://jupyter-notebook.readthedocs.io/en/stable/

PyPI (2022) rouge-score: Python Implementation of ROUGE Evaluation Metrics. Available At https://pypi.org/project/rouge-score/

Python Software Foundation, Python 3.13 documentation (2026). Available At: https://docs.python.org/3.13/

Sentence Transformers (2026) Sentence Transformers: Installation and Documentation. Available At https://www.sbert.net/docs/installation.html